

● AI ENGINEERING STACK

No One Knows These Questions

FOUNDATIONAL KNOWLEDGE

LLMOps Interview Q&A

Volume 1 • 20 Production-Grade Q&As

✓ MLOps vs LLMOps

✓ Prompt Drift

✓ Non-Determinism

✓ RAG Failures

✓ Agent Architectures

Swipe to master the 20 questions →

Q1. What is LLMOps and how does it differ from traditional MLOps?

LLMOps is the set of practices, tools, and workflows for deploying and operating LLM systems in production. It grew out of MLOps but the overlap is smaller than expected.

SHARED GROUND

- ✓ Both care about deployment pipelines, monitoring, versioning, & reliability.
- ✓ "Build something, ship carefully, watch in prod" applies to both.

NEW CHALLENGES

- ✗ Prompt versioning & CI/CD
- ✗ Non-deterministic outputs
- ✗ Context window management
- ✗ Per-token cost scaling

WHERE IT DIVERGES FUNDAMENTALLY

- **Different artifact:** In MLOps, you own/train the model. In LLMOps, models are external API infrastructure, not internal artifacts.
- **Text over code:** Highest-leverage variable is English prose prompts sitting in codebase, creating new governance needs.
- **Complex evaluation:** Can't evaluate helpfulness with precision/recall. Requires LLM judges, human raters, & behavioral rubrics.

 For more interview questions check link in caption.

Q2. What are your equivalent levers when quality drops, and why is retraining rarely first?

In classical ML, the answer is often "get more data and retrain." In LLMOps, retraining is the last resort. Leverage favors faster, cheaper interventions.

- 1 Prompt eng. (1st):** Fixes task rules & ambiguity. Takes hours, deploys instantly.
- 2 Retrieval imp. (2nd):** Better chunking & embeddings fix "model quality" bugs.
- 3 Few-shot (3rd):** Adding selected examples fixes specific failure patterns cheaply.
- 4 Model switch (4th):** Moving to a more capable model addresses capability gaps.
- 5 Fine-tuning (5th):** Justified only with hundreds of labeled failure examples and stable behavior change. Expensive, slow, creates artifact.

💡 WHY NOT RETRAIN FROM SCRATCH?

Most teams using external APIs don't even have access to retrain base models. For fine-tuning, time investment is only justified after exhausting cheaper options.

 For more interview questions check link in caption.

Q3. What does the full lifecycle of an LLM system look like? Where do failures occur?

The lifecycle is longer than most teams plan for, and failure modes cluster at predictable transition stages.

LIFECYCLE STAGES

- ✓ 1. Define testable
Scoping: terms
- ✓ 2. Iterative test case
Prompts: design
- ✓ 3. RAG, tools,
Integration: memory
- ✓ 4. **Offline** Establish
Eval: baselines
- ✓ 5. Canary & shadow
Rollout: mode
- ✓ 6. Quality, cost,
Monitoring: latency
- ✓ 7. **Iteration:** Steady-state tuning
- ✓ 8. Model
Deprecation: migrations

FAILURE CLUSTERS

- ✗ **At Deployment:** Built against happy-path eval set that doesn't reflect real user query variation.
- ✗ **Scaling Reliability:** No monitoring, small eval set, prompt fails at production volume.
- ✗ **Provider Updates:** Underlying model changes silently degrade quality without regression detection.

 For more interview questions check link in caption.

Q4. How does non-determinism affect system design, testing, and monitoring?

Non-determinism is the biggest conceptual shift from traditional engineering. Same inputs don't produce identical outputs.

SYSTEM DESIGN IMPACT

Can't treat outputs as deterministic function calls. Downstream JSON parsers and logic must tolerate phrasing/structural variation. For safety-critical paths (financial, access decisions), **set temperature to 0** to trade diversity for predictability.

TESTING IMPACT

No traditional unit tests asserting exact strings. Write **semantic assertion tests** ("mentions customer name", "includes citation", "no prohibited language"). For regressions, run test cases 5-10 times and measure pass rates (e.g., 70% reveals consistency bugs).

MONITORING IMPACT

Can't alert on "output changed from yesterday" because outputs always change. Monitor **distributions and statistical process control**: quality score shifts, refusal rate changes, and format compliance drops.

 For more interview questions check link in caption.

Q5. Why can't you use standard ML metrics to evaluate LLMs? What replaces them?

Standard ML metrics require a single correct answer. "Write a helpful summary" has thousands of valid answers and no canonical label.

✗ WHY STANDARD METRICS FAIL

- ✗ **Accuracy:** Requires canonical ground truth ("Explain ML to a 10-year-old" has none).
- ✗ **F1 / Precision / Recall:** Measure classification agreement, not complex language quality.
- ✗ **RMSE:** Measures numeric prediction error; irrelevant to prose.

✓ WHAT REPLACES THEM

- ✓ **Task rubrics:** Score accuracy, completeness, tone, and format independently.
- ✓ **LLM-as-judge:** Trusted model evaluates outputs against rubrics at production scale.
- ✓ **Human eval:** Gold standard ground truth to periodically calibrate automated evaluators.
- ✓ **Reference metrics:** BLEU/ROUGE/BERTScore for translation/summarization.
- ✓ **Behavioral metrics:** Task completion, retention, and conversion validate business utility.

 For more interview questions check link in caption.

Q6. Explain "prompt drift." How is it similar to and different from data drift?

Prompt drift occurs when user inputs gradually shift away from the distribution your prompt was designed for, degrading quality without code changes.

= SIMILARITY TO CLASSICAL DATA DRIFT

In classical ML, inference data shifts from training distributions over time. Similarly, prompts designed for launch traffic underperform as user behavior evolves, new user segments arrive, or edge cases become common.

≠ THE KEY DIFFERENCE

In classical ML, drift only happens to inputs while the model stays fixed. In LLMops, you face drift in inputs **AND** in underlying model behavior (due to provider updates), plus adversarial prompt injections!

🛡️ DETECTION & MANAGEMENT

- ✓ Track query embedding distributions over time to catch centroid shifts away from test sets.
- ✓ Run evaluation suites against production prompts monthly even if nothing changed intentionally.

 For more interview questions check link in caption.

Q7. What are the 4 versioning concerns in LLMOps, and what happens when they change?

Classical MLOps versions datasets and models. LLMOps has four distinct artifacts that require strict version control.

1. MODEL VERSION

Underlying API model (e.g., GPT-4-0409). Provider updates silently shift tone, formatting, or safety. Pin versions and run full regression before routing traffic.

2. PROMPT VERSION

System prompts & few-shot examples. Any edit can improve targeted cases while breaking others. Never deploy without running the full evaluation set.

3. RETRIEVAL CONFIG

Embeddings, chunking, and index parameters. Changes what context the model sees, altering outputs even if prompts and models remain identical.

4. EVALUATION SET

Test cases & rubrics. Changing test sets breaks historical comparability. Always document changes and maintain old sets for continuity.

 For more interview questions check link in caption.

Q8. Why is the LLM cost model fundamentally different from traditional ML hosting?

Traditional ML server hosting has mostly fixed compute costs. LLM APIs scale directly with token usage, creating variable and highly asymmetric costs.

\$ WHAT DRIVES COST

- **Variable output:** 2k-word response costs 10x more than 200 words.
- **System prompts:** Charged on every single request.
- **Chat history:** Grows linearly with turns.
- **RAG chunks:** Each query injects N chunks.
- **Model tiering:** GPT-4 is 10-20x pricier than cheap tiers for same request.

⚙️ HOW YOU CONTROL IT

- ✓ **Context caps:** Hard limits on history, chunks, & max output tokens.
- ✓ **Model routing:** Send simple queries to cheap models, complex to premium.
- ✓ **Caching:** Serve cached replies for identical/similar queries.
- ✓ **Prompt compression:** Keep system prompts lean; sentences cost money.
- ✓ **Feature attribution:** Monitor cost per feature.

 For more interview questions check link in caption.

Q9. What are the unique responsibilities of an LLMOps engineer?

Unique responsibilities cluster around the new artifacts, external dependencies, and probabilistic failure modes of LLMs.

PROMPT LIFECYCLE

Designing, versioning, testing, & deploying prompts as code; managing A/B tests and review workflows.

EVALUATION DESIGN

Building rubrics where there is no single correct answer; calibrating LLM-as-judge models against humans.

RETRIEVAL PIPELINE OPS

Maintaining ingestion pipelines, doc freshness, embedding migrations, & vector store performance.

COST & CONTEXT OPS

Monitoring token usage at system levels, diagnosing cost spikes, & optimizing context budgets.

VENDOR MANAGEMENT

Tracking model deprecations, evaluating releases, managing API rate limits, & planning migrations.

NON-DETERMINISM OPS

Building statistical pass/fail frameworks & behavioral assertions against output distributions.

 For more interview questions check link in caption.

Q10. How do regulatory and compliance requirements change with LLM systems?

LLM systems introduce complex legal and regulatory challenges around transparency, liability, and data privacy.



Explainability harder: LLMs lack decision rules. In regulated domains (FCRA, HIPAA), AI only informs decisions humans make & document.



Training data liability: Web data may contain copyrighted/personal info. GDPR right to erasure is difficult with model weights.



PII in prompts/logs: User prompts contain PII. Debugging logs become sensitive PII data stores requiring encryption & retention limits.



Hallucination liability: Confidently false advice in medical, legal, or finance creates severe liability without grounding & citations.



EU AI Act compliance: High-risk AI applications (hiring, credit, law enforcement) require strict documentation, transparency, and human oversight from day one.



For more interview questions check link in caption.

Q11. Describe the 4 major layers of an LLM system and where each fails in production.

The four layers—Model, Prompt, Retrieval, and Orchestration—fail in distinct ways with different operational signals.

🗣️ 1. MODEL LAYER

- ✗ Unannounced provider updates
- ✗ Latency spikes under load
- ✗ Rate limits & request queuing
- ✗ Sudden safety policy refusals

📋 2. PROMPT LAYER

- ✗ Degradation on real user queries
- ✗ Edge cases ignoring rules
- ✗ Adversarial prompt injections
- ✗ Ambiguous formatting rules

📁 3. RETRIEVAL LAYER

- ✗ Semantically similar but irrelevant chunks
- ✗ Phrasing mismatches missing info
- ✗ Stale or contradictory docs
- ✗ Bad chunk splitting strategy

🔄 4. ORCHESTRATION

- ✗ Infinite agent loops
- ✗ Unisolated session context mixing
- ✗ Missing retry logic on API errors
- ✗ Hanging requests without timeouts

📖 For more interview questions check link in caption.

Q12. What is the role of orchestration vs a simple API call wrapper?

A wrapper handles the happy path (20 lines of code). Orchestration is the robust software layer that makes production AI actually work.

🔧 WHAT ORCHESTRATION MANAGES

- ✓ **Multi-step sequencing:** Coordinating model calls, tools, & data lookups.
- ✓ **State management:** Maintaining chat history, tool results, & session state.
- ✓ **Tool calling:** Executing API/DB calls, validating, & feeding results back.
- ✓ **Error handling:** Retries, circuit breakers, & fallbacks for API failures.
- ✓ **Context window:** Pruning, summarizing, or dropping older context.
- ✓ **Routing & dispatch:** Classifying queries to specialized models or RAG.

🔍 CONTRAST WITH A WRAPPER

A wrapper is a simple function: take input, call API, return output. Orchestration manages happy paths, error paths, edge cases, retries, and state as a full software layer.

📖 For more interview questions check link in caption.

Q13. Why is the prompt layer treated as a first-class engineering artifact?

The prompt is the primary control surface defining behavior, rules, formatting, and tone. It has more influence on quality than any other component.

✔️ FIRST-CLASS PRACTICE

- ✓ **Version control:** Stored in source git repos, tracked, reviewable, reversible.
- ✓ **Testing gates:** Must pass automated evaluation suites before deployment.
- ✓ **Code reviews:** Required peer review because edits alter system behavior.
- ✓ **Staged rollout:** Deploy via dev → staging → prod canary pipelines.

❌ WHAT GOES WRONG WITHOUT IT

- ✗ Editing prompts in prod DBs causes silent regressions found via user complaints a week later.
- ✗ Duplicated system prompts across features become inconsistent when safety rules change.
- ✗ Unversioned prompts failing after model updates leave teams unable to debug root causes.

📖 For more interview questions check link in caption.

Q14. How does retrieval layer quality directly affect model output?

The model generates responses based primarily on what retrieval provides. The generation quality ceiling is set directly by retrieval quality.

🔗 THE DIRECT DEPENDENCY

The model can only work with what is inside its context window. Wrong context = wrong answer. No relevant context = hallucination or unhelpful "I don't know" replies.

🚩 HOW RETRIEVAL FAILURES MANIFEST

- **Recall failure (missing doc):** Model lacks required facts; either refuses or hallucinates plausible text.
- **Precision failure (irrelevant doc):** Confusing/contradictory context causes model to synthesize misleading blended answers.
- **Stale doc retrieval:** Model confidently answers based on outdated policy.

⚠️ WHY IT'S EASY TO MISS

Retrieval errors look like "model hallucinations" if you only look at outputs. You **MUST** log retrieved chunk IDs and scores alongside responses to diagnose root causes!

📖 For more interview questions check link in caption.

Q15. Trace the execution path end-to-end for a single user request across all 4 layers.

Example: Enterprise assistant query: "What's our policy on parental leave for part-time employees?"

- 1 Orchestration Input:** Receives request, checks auth/rate limits, loads chat history, & routes to RAG.
- 2 Retrieval Processing:** Expands synonyms, embeds query, vector searches top-k chunks, & filters ACLs.
- 3 Prompt Assembly:** Combines system prompt, query, history, & retrieved document chunks with citations.
- 4 Model Inference:** Model synthesizes policy chunks into a coherent, accurate answer citing source docs.
- 5 Post-Processing:** Validates format/citations, logs response, updates history, & delivers answer to user.
- 6 Async Monitoring:** Asynchronously runs quality evaluators, computes token costs, & checks anomaly detectors.

📌 For more interview questions check link in caption.

Q16. Architectural & operational differences: Chatbot vs RAG vs Agent?

These represent increasing architectural complexity. Operational surface area and incident complexity roughly double at each step!

1. CHATBOT (CONTAINED)

Arch: Prompt + history + message → model → response.

Ops Challenges: Prompt & history management, non-determinism. Contained failure surface (model behavior or prompt quality).

2. RAG SYSTEM (RETRIEVAL ADDED)

Arch: Adds embedding, vector database search, and context retrieval.

Ops Challenges: Ingestion pipeline reliability, document freshness, chunking maintenance, embedding versioning. Requires tooling to isolate retrieval vs generation errors.

3. AGENT (TOOLS & LOOPS ADDED)

Arch: Adds tool execution, multi-step decision loops, and branching.

Ops Challenges: Tool reliability, infinite loop prevention, partial task failure handling, runaway token cost control, multi-step trace observability.

 For more interview questions check link in caption.

Q17. Why do RAG systems introduce retrieval errors? How do you isolate root causes?

A chatbot has 1 failure mode (bad generation). RAG has 2: bad generation AND bad retrieval context. They look identical from the outside!

WHY RETRIEVAL FAILURES ARE NON-OBVIOUS

The model generates confidently from whatever context it receives, without flagging that the retrieved context itself was irrelevant or incorrect.

3-STEP ISOLATION PROCESS

- ✔ **Step 1: Log what was retrieved.** Foundation of RAG debugging. Must log retrieved chunk IDs and similarity scores alongside every response.
- ✔ **Step 2: Manual injection test.** Bypass retrieval by manually injecting correct knowledge chunks into prompt. If response is correct → retrieval failed. If still wrong → generation failed.
- ✔ **Step 3: Dedicated retrieval eval.** Evaluate recall@k independently of generation quality to fix chunking, embeddings, or parameters.

 For more interview questions check link in caption.

Q18. Agents can call tools, loop, and make decisions. What new risks does this introduce?

Each autonomous agent capability introduces severe operational risks that single-turn LLM calls do not have.

⚠ CORE RISK CLASSES

- ✗ **Tool risks:** Real-world actions (emails, DB edits) can cause irreversible harm if manipulated. API failures break multi-step tasks.
- ✗ **Looping risks:** Indefinite loops when termination conditions aren't met burn tokens continuously and trigger repeated actions.
- ✗ **Compound errors:** A slight error at step 2 leads to a completely unintended result by step 10.

🛡 NEW OPS REQUIREMENTS

- ✓ **Hard iteration limits:** Cap max tool calls & reasoning steps per task.
- ✓ **Persistent state logging:** Full execution tracing for debugging.
- ✓ **Human-in-the-loop:** Mandatory approval checkpoints for irreversible actions.
- ✓ **Session cost alerts:** Catch runaway agents instantly.
- ✓ **Loop detection:** Heuristics based on action history.

 For more interview questions check link in caption.

Q19. How does context window management differ across chatbot, RAG, and agent?

Deciding what information to keep in fixed-size context windows scales dramatically in challenge as system complexity grows.

CHATBOT (HISTORY MANAGEMENT)

Fills with conversation history. Simple management: keep last N turns or summarize older turns into a running summary. System prompt + current message always get priority.

RAG SYSTEM (BUDGET-AWARE RETRIEVAL)

Contains history + multiple retrieved chunks + query. Must re-rank chunks, limit count to prevent overflow, and balance retrieved content against conversation history. Retrieval must be context-budget-aware.

AGENT (MULTI-STEP ACCUMULATION)

Accumulates tool schemas, tool results from previous steps, history, and reasoning. Can become enormous in 10-step tasks. Strategies: tool description pruning, result summarization, and periodic progress summarization.

 For more interview questions check link in caption.

Q20. For customer support, how would you decide between chatbot, RAG, or agent?

The decision comes down to actual business requirements and risk tolerance, not which technology sounds most impressive.

☰ KEY DECISION CRITERIA

- **Look up specific company info?** Policies & specs require RAG (chatbots hallucinate or give generic answers).
- **Take real-world actions?** Checking orders, processing returns, or updating CRMs require Agents with tool access.
- **Tolerance for errors?** As error stakes increase (wrong return vs bad text), human-in-the-loop validation becomes critical.
- **Query complexity?** 90% simple FAQs → RAG. Frequent multi-step resolutions → Agent reasoning.

💡 PRACTICAL RECOMMENDATION

Start with RAG. It handles the majority of support queries well without the operational complexity of agents. Add tool-calling incrementally only as specific action requirements justify the complexity!

 For more interview questions check link in caption.